

Poisson 0.12 req/s 三种调度策略综合报告

本报告按 0.15 req/s 完整报告的口径，统一比较 FCFS、SJF 与 RH-SAIL；0.12 是主实验，0.15 仅作为压力区间参照。

技术结论

- 三种策略均完成 1400/1400，成功率 100%，未发现 OOM、context length、KV cache、CUDA 或 traceback 错误。
- 0.12 下没有一个新策略全面优于 FCFS。FCFS 的平均完成时间不是最低，但系统完成时间、吞吐、调度开销和极端长尾最均衡；最大完成时间只有 395.8s。
- SJF 优化的是平均/中位完成时间，不是稳健性。它把平均等待降低 60.4%、平均完成时间降低 15.1%，但 P99 完成时间变为 FCFS 的 4.03x，最大完成时间变为 13.47x。这说明少量长 DAG 被短任务持续越过。
- RH-SAIL 在 0.12 下没有超过 FCFS。相对 FCFS，其 makespan +3.26%、总 token 吞吐 -2.29%、平均完成时间 +47.06%，调度开销达到 11.42%。
- RH-SAIL 的可验证价值是修复 SJF 的极端 service 长尾：相对 SJF，P99 service 降低 72.6%、最大 service 降低 93.5%、P99 完成时间降低 45.0%；但它仍未在这些长尾指标上击败 FCFS。
- 实际最后一次请求约在 192.8 分钟到达，三种策略仅需再排空 4.3–10.5 分钟；相比 0.15 的 32.5–39.5 分钟尾部，0.12 是接近容量拐点但仍可稳定排空的负载。

核心系统性能

完成时间与吞吐

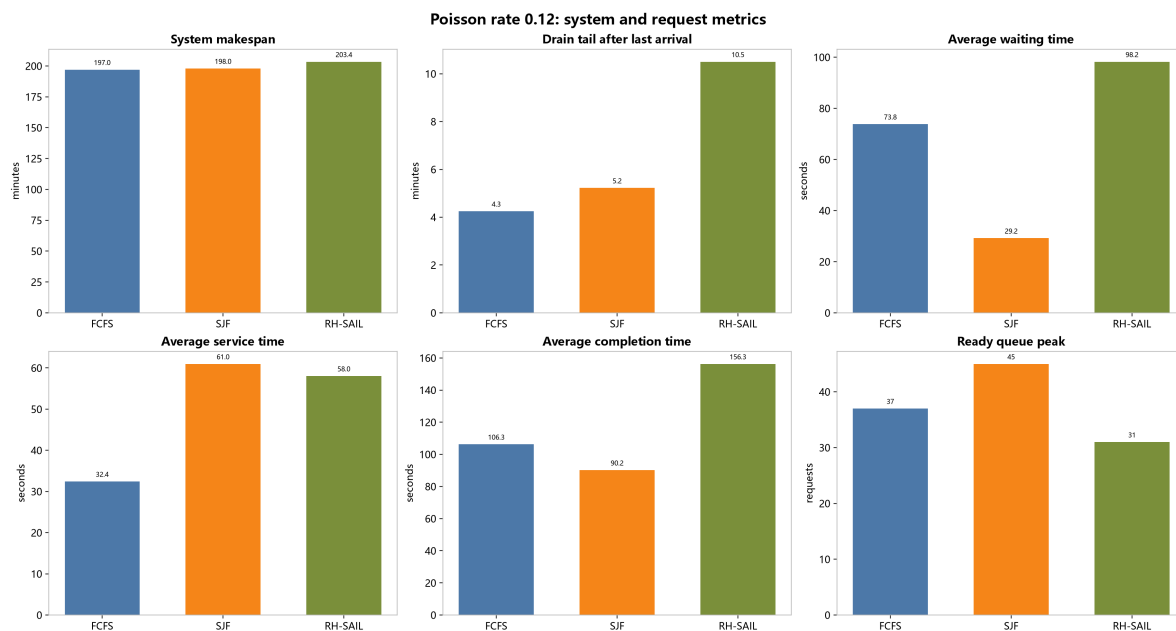
策略	λ (req/s)	完成	Makespan (min)	Tokens (M)	Req TP (req/s)	Total TP (tok/s)	Output TP (tok/s)
FCFS	0.12	1400/1400	197.0	28.124	0.1184	2379.3	261.7
FCFS	0.15	1400/1400	192.9	28.518	0.1210	2464.1	275.7
SJF	0.12	1400/1400	198.0	28.356	0.1179	2387.2	261.9
SJF	0.15	1400/1400	186.7	28.368	0.1250	2532.9	278.5
RH-SAIL	0.12	1400/1400	203.4	28.376	0.1147	2324.8	255.1
RH-SAIL	0.15	1400/1400	193.8	28.262	0.1204	2429.9	268.9

到达、积压与调度开销

策略	λ (req/s)	到达结束 (min)	排空尾部 (min)	Ready Queue avg / max	调度开销 (s / %)
FCFS	0.12	192.8	4.3	2.7 / 37	444.9 / 3.76%
FCFS	0.15	154.2	38.7	96.3 / 237	321.5 / 2.78%
SJF	0.12	192.7	5.2	4.6 / 45	442.4 / 3.72%
SJF	0.15	154.2	32.5	54.1 / 131	296.8 / 2.65%
RH-SAIL	0.12	192.9	10.5	2.3 / 31	1394.3 / 11.42%
RH-SAIL	0.15	154.4	39.5	10.3 / 33	1448.1 / 12.45%

Makespan 同时受到请求到达窗口影响，因此不能直接用 0.12 与 0.15 的数值判断负载优劣；排空尾部 = Makespan - 实际最后到达时间 更能反映积压。Ready Queue avg / max 是调度器候选队列采样，RH-SAIL 会先做 admission control，所以它的 max 不能视为系统总 backlog。

0.12 三次运行的总 token 数离散度为 0.90%。因此 SJF 相对 FCFS 仅 +0.33% 的 token/s 差异，不能单独归因于调度；它同时生成了更多 token。goodput_tokens_per_second 在当前实现中等同于总 token 吞吐，不代表回答质量。



请求完成时间与长尾

等待时间 是请求到达至首个 op 开始；service time 是首个 op 开始至 DAG 完成；完成时间 = 等待时间 + service time。

策略	平均等待	平均 service	P50 service	P95 service	P99 service	最大 service	平均完成	P99 完成	最大完成
FCFS	73.8s	32.4s	23.9s	86.8s	133.2s	206.2s	106.3s	312.3s	395.8s
SJF	29.2s	61.0s	24.4s	156.2s	737.2s	5301.3s	90.2s	1259.2s	5329.9s
RH-SAIL	98.2s	58.0s	42.7s	156.1s	201.8s	345.7s	156.3s	693.1s	4681.0s

这张表揭示了三个目标之间的冲突：

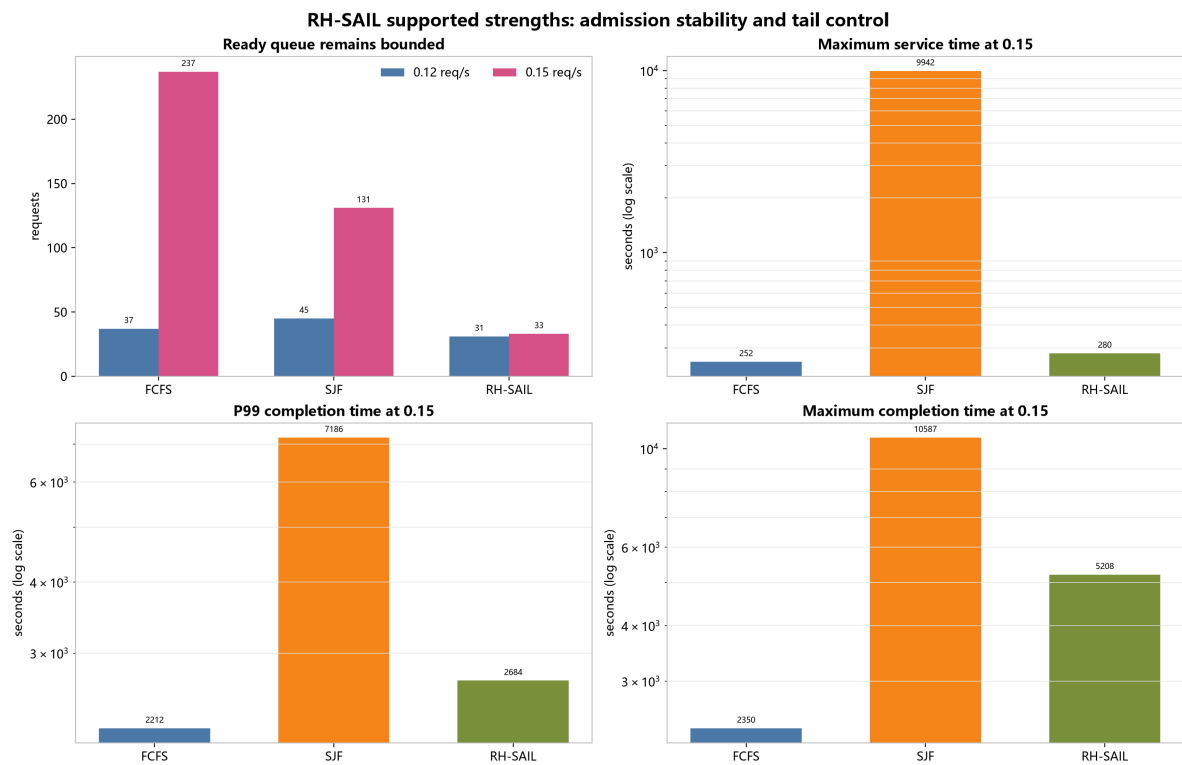
1. FCFS 保持请求级顺序，平均等待高于 SJF，但已启动请求很少被长期搁置，因此 service 与最大完成时间最稳。
2. SJF 按初始输入长度优先短请求，显著降低平均值；代价是 swebench_verified 等长 DAG 可能被反复越过，形成 5301.3s 最大 service。
3. RH-SAIL 用 admission、progress commitment 和 emergency guard 限制这种饿死，但 admission 等待和 rollout 成本又抬高了整体完成时间。

请求连续性

请求的 service 窗口被拆成实际有 op 运行的 active wall time 与没有该请求 op 运行的 dormant time。该口径直接衡量“请求启动后是否被搁置”。

策略	平均 active wall	平均 dormant	Dormant fraction	P95 最大 op 间空档	P95 service stretch
FCFS	28.4s	4.1s	15.6%	12.3s	1.96x
SJF	29.7s	31.3s	19.2%	52.2s	3.22x
RH-SAIL	31.4s	26.7s	38.4%	65.8s	4.91x

SJF 的平均 dormant fraction 不高，但 P95 gap 与最大 service 很大，说明问题集中在少量长请求，而不是所有请求都变慢。RH-SAIL 把最极端的 service 拉回，但平均 dormant fraction 上升到 38.4%，表明当前参数下的连续性保护仍不足以抵消 admission/rollout 带来的停顿。



调度、队列与 DAG 指标

策略	Ready avg / peak	Dependency stall	调度开销	Critical path	DAG parallelism	跨 device 依赖	并行利 用率
FCFS	2.7 / 37	13.8s	444.9s (3.76%)	26.2s	1.317	5.18	86.24%
SJF	4.6 / 45	79.9s	442.4s (3.72%)	26.7s	1.309	5.53	86.77%
RH-SAIL	2.3 / 31	55.6s	1394.3s (11.42%)	26.7s	1.315	5.95	84.67%

必须注意：ready_queue 是进入各调度器候选集合之后的采样。FCFS/SJF 基本暴露全部 ready work；RH-SAIL 会先做 active-workflow admission，因此其 31 的 peak 只表示已接纳候选池有界，不能解释为系统总 backlog 只有 31，也不能直接与 FCFS 的 37 作同义比较。

RH-SAIL 在 0.12/0.15 分别记录 8 / 54 次 admission throttle，active DAG limit 为 15。这证明 admission 机制确实生效，但也说明未接纳请求的等待需要单独计入端到端指标；最终是否更好应以 completion time、tail 和吞吐判断，而不是仅看 ready queue。

相对 FCFS 的变化

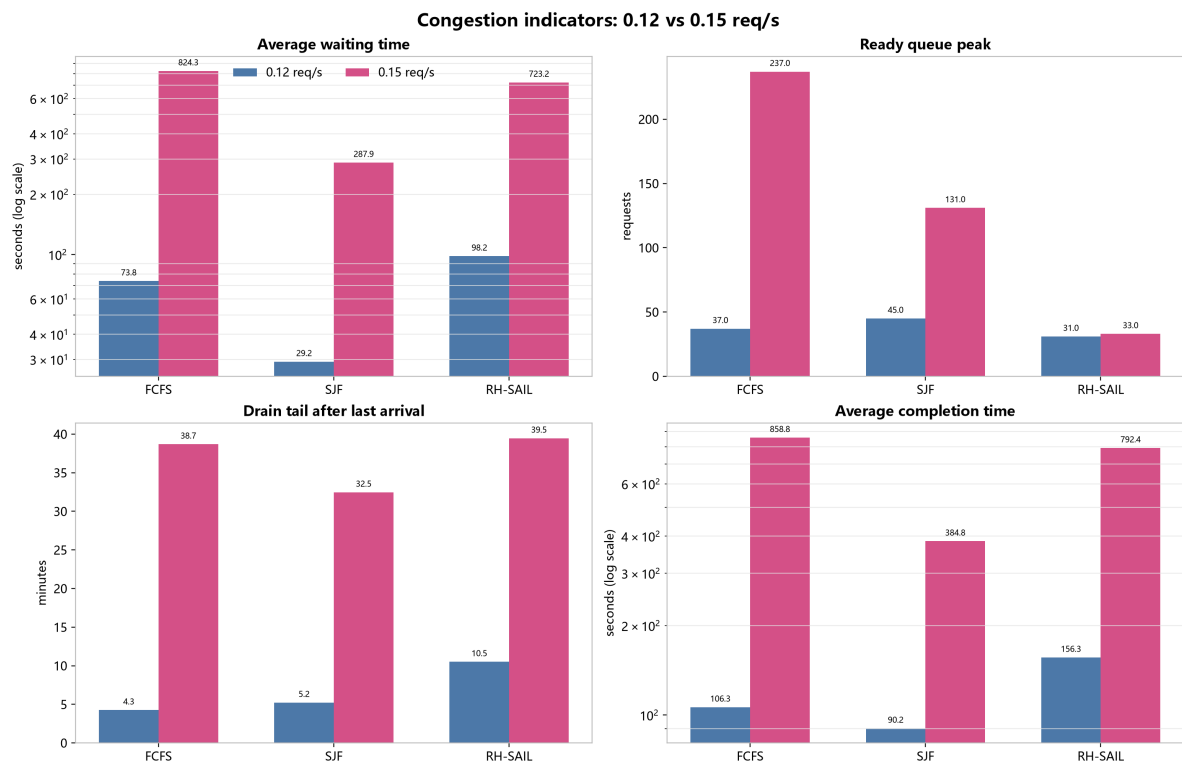
指标	优选方向	SJF vs FCFS	RH-SAIL vs FCFS
系统总完成时间	越低越好	+0.49%	+3.26%
总 token 吞吐	越高越好	+0.33%	-2.29%
平均等待	越低越好	-60.44%	+33.05%
平均 service	越低越好	+87.98%	+78.95%
平均完成时间	越低越好	-15.14%	+47.06%
P95 service	越低越好	+79.95%	+79.91%
P99 完成时间	越低越好	+303.18%	+121.92%
最大完成时间	越低越好	+1246.68%	+1082.72%
调度开销	越低越好	-0.56%	+213.40%

百分比按 $(\text{策略值} / \text{FCFS} - 1) \times 100\%$ 计算。低负载下，SJF 形成“平均值更好、尾部更差”的 Pareto 点；RH-SAIL 当前没有形成相对 FCFS 的 Pareto 改进，因为吞吐、平均值、尾部和开销至少有一项同时变差。

与 0.15 压力区间综合比较

策略	平均等待 0.12 / 0.15	平均完成 0.12 / 0.15	排空尾部 0.12 / 0.15	Token/s 0.12 / 0.15	Device busy 0.12 / 0.15
FCFS	73.8s / 824.3s	106.3s / 858.8s	4.3min / 38.7min	2379.3 / 2464.1	86.2% / 91.7%
SJF	29.2s / 287.9s	90.2s / 384.8s	5.2min / 32.5min	2387.2 / 2532.9	86.8% / 91.6%
RH-SAIL	98.2s / 723.2s	156.3s / 792.4s	10.5min / 39.5min	2324.8 / 2429.9	84.7% / 88.9%

- 0.12 的到达窗口约 192.8min，0.15 的到达窗口约 154.2min；不能只用 makespan 跨速率比较，因为低到达速率本身会拉长实验。
- 从 0.12 提高到 0.15 后，device busy 只增加约 4-5 个百分点，token/s 增幅也有限，但等待和排空尾部成倍增长，说明 0.15 已超过有效服务能力。
- 0.15 下 SJF 的平均完成时间仍最好，但长 DAG 尾部进一步恶化；RH-SAIL 相对 SJF 继续压低极端长尾，但相对 FCFS 仍不是全面胜出。



各数据集平均 Service Time

每个数据集均有 200 个请求。数据集差异同时包含 DAG 结构、prompt 长度和生成长度影响。

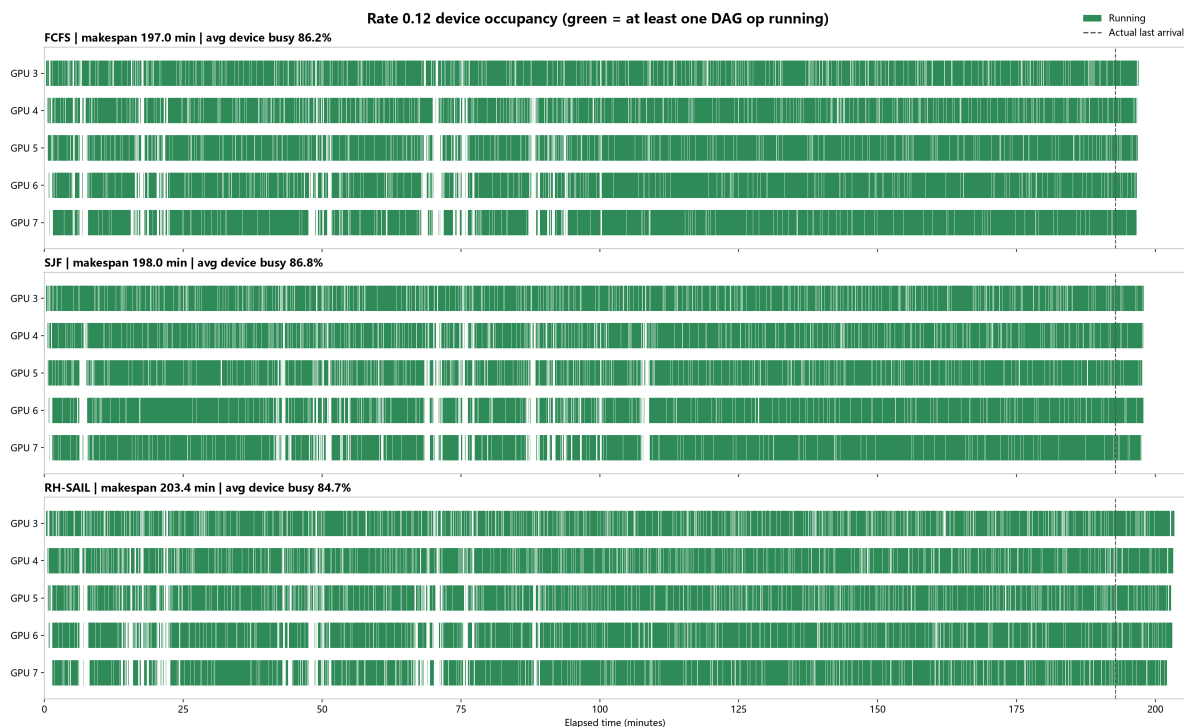
Dataset	FCFS	SJF	RH-SAIL
gsm8k	9.6s	8.0s	11.5s
strategyqa	17.7s	15.7s	30.3s
mmlu_pro	22.9s	22.5s	26.1s
math	41.0s	42.5s	73.5s
mbpp	33.9s	35.0s	58.6s
hotpotqa	26.0s	49.7s	63.4s
swebench_verified	76.0s	253.5s	142.9s

SJF 的长尾主要集中在长工作流：对短任务它能快速清空，但对 `swebench_verified` 等长 DAG，平均 service 会显著上升。RH-SAIL 缓和了这一问题，却使多个中等 DAG 的 service 高于 FCFS。

Device 拥挤与 GPU 使用

0.12 Device 占用

绿色表示该 device 至少有一个 DAG op 正在运行，虚线表示实际最后一次请求到达。三种策略在到达窗口内均保持较高占用，尾部远短于 0.15。

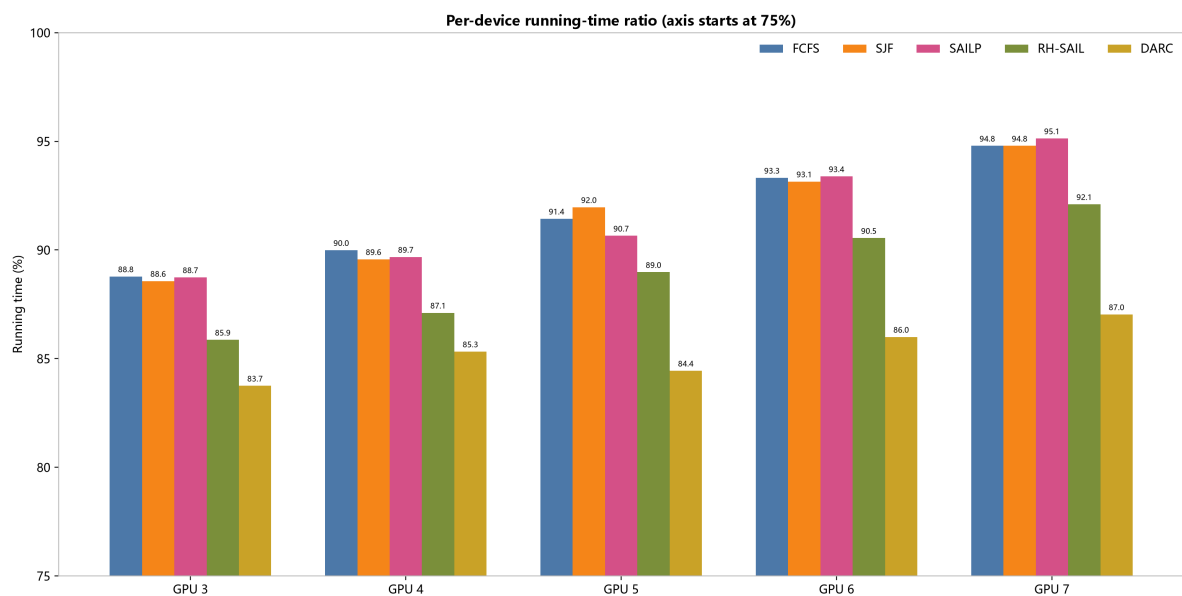


0.15 Device 拥挤

0.15 压力图中的竖直虚线是理论泊松到达窗口末端。虚线以后仍持续运行的绿色区间就是系统排空积压的尾部。该图来自完整五策略报告；本报告重点观察 FCFS、SJF 与 RH-SAIL 三行，SAILP、DARC 仅作为原图背景保留。



Physical GPU	FCFS busy (%)	SJF busy (%)	RH-SAIL busy (%)
GPU 3	88.8	88.6	85.9
GPU 4	90.0	89.6	87.1
GPU 5	91.4	92.0	89.0
GPU 6	93.3	93.1	90.5
GPU 7	94.8	94.8	92.1

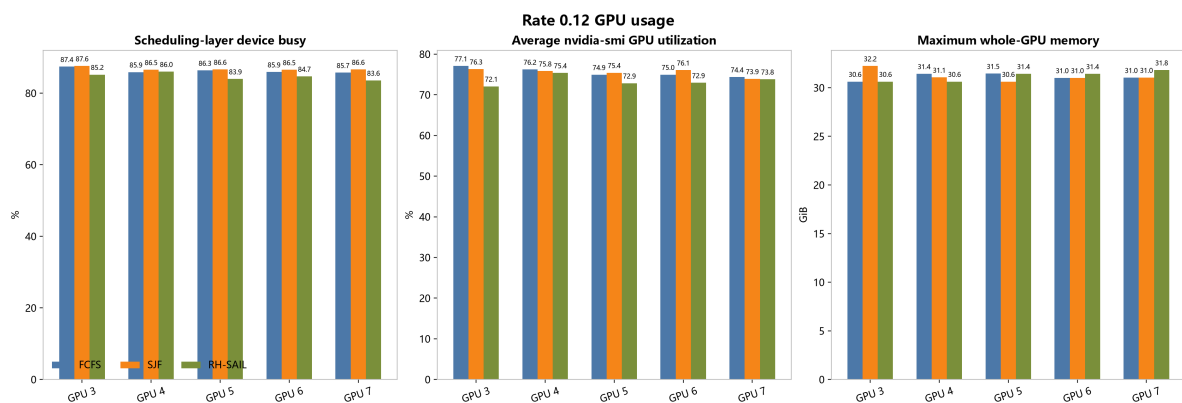


0.15 下 FCFS/SJF 的平均 device busy 已达到约 91.6%，RH-SAIL 为 88.9%。三者在请求停止到达后仍持续运行约 32-40 分钟，说明主要问题是到达速率超过有效服务能力，而不是 GPU 长时间整体空闲；RH-SAIL 更低的 busy 还叠加了 admission 与 rollout 开销。

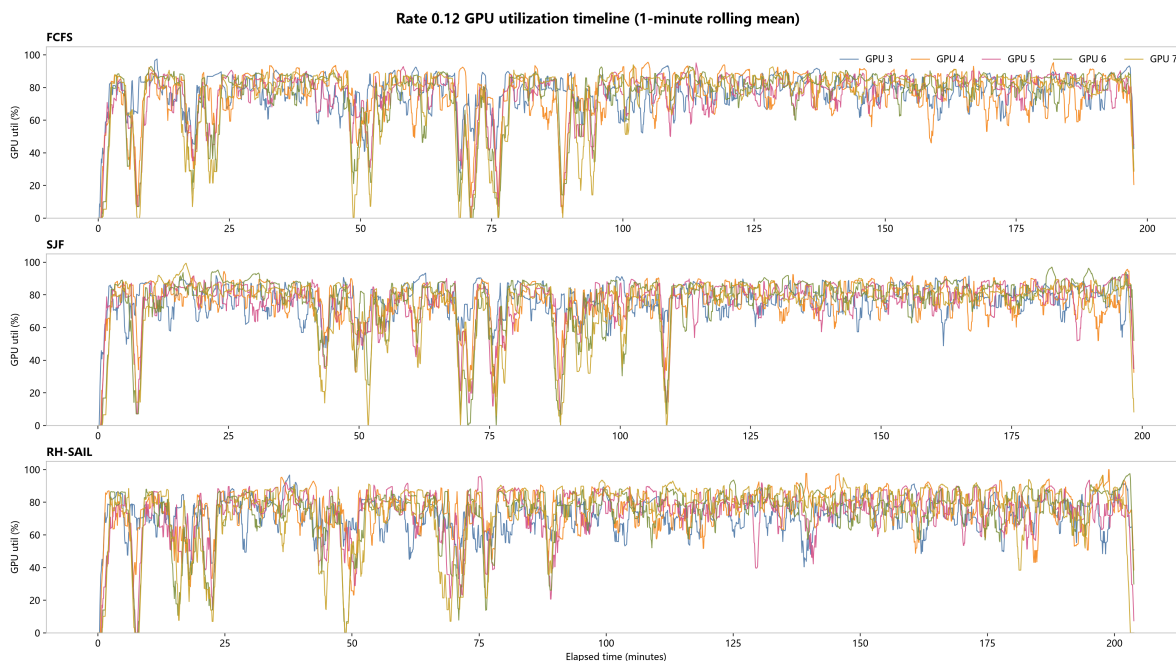
0.12 GPU 硬件采样

调度层 busy 来自 op 的真实 start/end；nvidia-smi utilization、显存和功耗来自每 5 秒采样，两者不是同一口径。0.15 的旧三策略没有完整连续的同口径 nvidia-smi CSV，因此这里不伪造 0.15 硬件瞬时利用率对比。

策略	平均 GPU util	平均 P95 util	单卡最高显存	平均单卡功耗
FCFS	75.5%	92.2%	31.45GiB	244.2W
SJF	75.5%	92.2%	32.23GiB	247.2W
RH-SAIL	73.4%	92.6%	31.82GiB	240.7W



时间线使用 1 分钟滚动平均。泊松到达会形成短暂空档，但没有长期全卡空闲。



综合判断与更合适的优化目标

当前结果不支持“寻找一个在所有指标上击败 FCFS 的排序规则”。更合理的是把目标明确成多目标或带约束优化：

- 主目标：最小化平均或 P95 completion time**，直接对应用户看到的端到端延迟。
- 硬约束：限制 P99/max service stretch 或最大 inter-op gap**，防止 SJF 式长 DAG 饿死。
- 吞吐 guardrail：token/s 不低于 FCFS 的 98%–99%**，避免用大量调度计算换取很小的延迟收益。
- 开销 guardrail：scheduler overhead 控制在 3%–5%**；当前 RH-SAIL 的 11%–12% 明显过高。
- 负载分区：0.12 使用轻量 FCFS/SJF hybrid，只有检测到持续 backlog 后才启用 admission/rollout**。RH-SAIL 更像高压保护模式，而不是所有负载下的默认策略。

按这个目标，下一步最有价值的基线不是继续堆复杂打分项，而是实现一个**带 aging/最大 gap 约束的 SJF**：正常时按预测剩余工作量排序，超过等待或 gap 阈值的请求强制提升优先级。它直接保留 SJF 的均值优势，同时针对本实验最明显的失败模式。

实验配置

项目	设置
数据	7 个数据集 × 200，共 1,400 请求
调度器	FCFS、SJF、RH-SAIL
到达过程	Poisson，0.12 req/s，batch size 1，seed 20260709
模型	Llama-3.1-8B-Instruct，real vLLM
GPU	A800 physical GPU 3–7，共 5 张
上下文	max model len 32768，每个 op 最多输出 2048 tokens
显存参数	gpu-memory-utilization=0.75
Prefix caching	关闭

方法、限制与复现

- 每个配置仅运行一次，当前结论是描述性结果，尚无重复实验置信区间。
- 解码温度与并发执行会带来生成非确定性；三次运行总 token 数存在轻微差异，token/s 必须结合总 token 数解释。
- 0.12 与 0.15 使用相同数据顺序和 arrival seed，但请求到达时间按速率缩放。
- 每种策略只有一次运行，1%~3% 的差异不能视为稳定排序；正式结论至少需要 3 个 arrival/generation seed。
- Prefix caching 关闭，因此当前实验不能验证 SAIL 论文中的真实 KV 状态亲和收益。
- 本报告的 service、continuity、dataset 指标由 1,400 条请求级 JSON 重算；GPU 指标按 runner 时间窗口切分硬件采样。

数据文件

- 汇总指标: `analysis/rate_metrics.csv`
- 分数据集指标: `analysis/rate012_dataset_metrics.csv`
- GPU 硬件汇总: `analysis/rate012_gpu_hardware.csv`
- GPU 采样切片: `analysis/rate012_gpu_samples.csv`
- 基础指标与图表生成脚本: `analysis/generate_rate012_report.py`
- 综合报告校验脚本: `analysis/validate_rate012_report.py`